



Знания — лучшая инвестиция



orion11

16 сен 2025 в 19:30

Вышла Java 25



21 мин



44K

Java*

Вышла общедоступная версия [Java 25](#). В этот релиз попало около [2600 закрытых задач](#) и [18 JEP'ов](#). Release Notes можно посмотреть [здесь](#). Полный список изменений API — [здесь](#).

Java 25 является LTS-релизом, а значит у него будут выходить обновления [как минимум 5 лет](#) с момента выхода (до сентября 2030 года).

Скачать JDK 25 можно по этим ссылкам:

- [Oracle JDK](#) (лицензия [NFTC](#))
- [OpenJDK](#) (лицензия [GPLv2 with Classpath Exception](#))

Рассмотрим все JEP'ы, которые попали в Java 25.

Язык

Module Import Declarations (JEP 511)

Module Import Declarations, которые были в режиме preview в [Java 23](#) и [Java 24](#), стали постоянной языковой конструкцией. По сравнению с Java 24 фича осталась без изменений.

Декларация `import module M` эквивалентна импорту всех экспортированных пакетов из модуля `M` и его транзитивных зависимостей в текущий модуль.

Например, импорт модуля `java.base` имеет тот же эффект, как если бы мы вручную импортировались все его 54 экспортированных пакета:

```
import java.io.*;
import java.lang.*;
import java.lang.annotation.*;
// ... 49 packages ...
import javax.security.auth.x500.*;
import javax.security.cert.*;
```

[Объяснить с](#)

Таким образом, написав всего лишь один импорт, можно будет получить доступ до таких неотъемлемых классов и интерфейсов как `List`, `Map`, `Stream`, `Path`, `Function` и др. без необходимости отдельного импорта их с указанием соответствующих пакетов.

Такое нововведение может быть полезным при прототипировании, изучении языка и новых фич, а также для написания коротких скриптов, которые запускаются напрямую [без предварительной компиляции](#).

При использовании *компактных исходных файлов* модуль `java.base` импортируется автоматически. Об этом следующий JEP 512.

Compact Source Files and Instance Main Methods (JEP 512)

Компактные исходные файлы и instance-методы `main()` стали постоянными. Среди рассматриваемых сегодня эта языковая фича была в preview дольше всех, появившись ещё аж в прошлом LTS-релизе: [Java 21](#), [Java 22](#), [Java 23](#) и [Java 24](#).

В этом релизе есть несколько изменений:

- Название "простые исходные файлы" поменялось на "компактные исходные файлы".
- Класс `java.io.IO` теперь стал `java.lang.IO`. Значит, он теперь импортируется неявно в любой исходный файл (необязательно компактный).
- Статические методы класса `IO` больше не импортируются неявно в компактных исходных файлах. Теперь придётся указывать класс (например, `IO.println("Hello, world!")`), либо вручную статически импортировать методы.
- Реализация класса `IO` теперь базируется на `System.out` и `System.in`, а не на `java.io.Console`.

Какие возможности даёт JEP 512?

Теперь можно запускать классы, у которых метод `main()` не является `public static` (т.е. является instance-методом) и у которого нет параметра `String[] args`:

```
class HelloWorld {  
    void main() {  
        System.out.println("Hello, World!");  
    }  
}
```

[Объяснить](#) с 

В таком случае во время запуска JVM сама создаст экземпляр класса `HelloWorld` и вызовет у него метод `main()`:

```
$ java HelloWorld.java  
Hello, World!
```

[Объяснить](#) с 

Кроме того, можно запускать файлы и без объявленного класса вовсе. Такие файлы называются *компактными исходными файлами*:

```
// HelloWorld.java

String greeting = "Hello, World!";

void main() {
    System.out.println(greeting);
}
```

[Объяснить с](#) 

```
$ java HelloWorld.java
Hello, World!
```

[Объяснить с](#) 

В таком случае виртуальная машина сама объявит неявный класс, в который поместит метод `main()` и другие верхнеуровневые объявления в файле:

```
// class <some name> { ← неявно
String greeting = "Hello, World!";

void main() {
    System.out.println(greeting);
}
// }
```

[Объяснить с](#) 

Неявный класс обладает практически всеми возможностями явного класса (возможность содержать методы, поля), но есть несколько отличий:

- Неявный класс может находиться только в безымянном пакете.
- Код в неявном классе не может ссылаться на него по имени.
- Неявный класс всегда имеет один дефолтный конструктор без аргументов.
- Неявный класс всегда является `final` и наследуется от `java.lang.Object`.

При этом неявный класс не является безымянным: у него есть **имя**, совпадающее с именем файла (но это является деталью реализации, на которую не стоит полагаться).

Наконец, с помощью класса `java.lang.IO` можно в примере выше заменить магический `System.out.println()` на более короткий и понятный `IO.println()`:

```
String greeting = "Hello, World!";

void main() {
    IO.println(greeting);
}
```

[Объяснить с](#) 

Ещё одной особенностью компактных исходных файлов является то, что каждый такой файл неявно импортирует модуль `java.base` (фича из предыдущего JEP 511). Это значит, что ко всем базовым классам Java можно обращаться без необходимости импортов:

```
void main() {  
    IO.println(List.of("James", "Bill", "Guy"));  
}
```

[Объяснить с](#) 

Компактные исходные файлы и instance-методы `main()` вводятся в Java с двумя целями:

- Облегчить процесс обучения языку. На новичка, только что начавшего изучение Java, не должно сваливаться всё сразу, а концепции должны вводиться постепенно, начиная с базовых (переменные, циклы, процедуры) и постепенно переходя к более продвинутым (классы, области видимости).
- Облегчить написание коротких программ и скриптов. Количество церемоний для них должно быть сведено к минимуму.

Flexible Constructor Bodies (JEP 513)

Flexible Constructor Bodies, которые были в preview три релиза ([Java 22](#), [Java 23](#), [Java 24](#)), стали постоянной фичей языка. По сравнению с Java 24 изменений нет.

Flexible Constructor Bodies разрешают писать инструкции кода в конструкторе перед явным вызовом конструктора (`super()` или `this()`):

```
public class PositiveBigInteger extends BigInteger {  
    public PositiveBigInteger(long value) {  
        if (value <= 0)  
            throw new IllegalArgumentException("non-positive value");  
        super(value);  
    }  
}
```

[Объяснить с](#) 

Напомним, что с самого первого релиза Java 1.0 это было запрещено, поэтому в случаях, когда необходимо выполнить код перед вызовом конструктора, приходилось использовать обходные пути, например, прибегать к вспомогательным статическим методам:

```
public class PositiveBigInteger extends BigInteger {  
    public PositiveBigInteger(long value) {  
        super(verifyPositive(value));  
    }  
  
    private static long verifyPositive(long value) {  
        if (value <= 0)  
            throw new IllegalArgumentException("non-positive value");  
        return value;  
    }  
}
```

```
}
```

[Объяснить с](#) 

Или к вспомогательным конструкторам, если нужно передать одно и то же значение для нескольких параметров:

```
public class Super {  
    public Super(C x, C y) { ... }  
}  
  
public class Sub extends Super {  
    private Sub(C x) { // Auxiliary constructor  
        super(x, x); // x is shared here  
    }  
  
    public Sub(int i) {  
        this(new C(i));  
    }  
}
```

[Объяснить с](#) 

В Java 25 то же самое можно реализовать гораздо короче:

```
public class Sub extends Super {  
    public Sub(int i) {  
        var x = new C(i);  
        super(x, x);  
    }  
}
```

[Объяснить с](#) 

Не всякий код можно поместить перед вызовом конструктора: код в прологе не должен ссылаться на конструируемый объект (читать поля, вызывать instance-методы). Рассмотрим несколько примеров некорректного кода:

```
class A {  
    int i;  
  
    A() {  
        System.out.print(this); // Error  
        var x = i;               // Error  
        hashCode();              // Error  
        super();  
    }  
}
```

[Объяснить с](#) 

Ссылаться на родительский объект также нельзя (ведь это тоже часть текущего объекта):

```
class B {
    int i;
    void m() {}
}

class C extends B {
    C() {
        var x = i; // Error
        m();        // Error
        super();
    }
}
```

[Объяснить с](#) 

Также запрещены ситуации, когда есть неявная ссылка на объект, например, через экземпляр внутреннего класса:

```
class Outer {
    class Inner {
    }

    Outer() {
        new Inner(); // Error
        super();
    }
}
```

[Объяснить с](#) 

Однако если читать поля конструируемого класса до вызова `super()` нельзя, то инициализировать их можно:

```
class A {
    int i;

    A(int i) {
        this.i = i; // OK
        super();
    }
}
```

[Объяснить с](#) 

Это может быть полезным для ситуаций, когда в конструкторе суперкласса может случайно прочитаться нежелательное дефолтное значение поля при вызове виртуального метода:

```
class Super {
    Super() { overriddenMethod(); }

    void overriddenMethod() {
        System.out.println("hello");
    }
}
```

```
    }  
}  
  
class Sub extends Super {  
    final int x;  
  
    Sub(int x) {  
        this.x = x;  
    }  
  
    @Override  
    void overriddenMethod() {  
        System.out.println(x); // new Sub(42) will print 0  
    }  
}
```

[Объяснить с](#) 

Чтобы предотвратить такую ситуацию, нужно поместить инициализацию поле выше вызова `super()` :

```
class Super {  
    Super() { overriddenMethod(); }  
  
    void overriddenMethod() {  
        System.out.println("hello");  
    }  
}  
  
class Sub extends Super {  
    final int x;  
  
    Sub(int x) {  
        this.x = x;  
        super();  
    }  
  
    @Override  
    void overriddenMethod() {  
        System.out.println(x); // new Sub(42) will print 42  
    }  
}
```

[Объяснить с](#) 

Также инициализация полей до `super()` можно пригодиться в проекте Valhalla для [definite assignment](#) полей `null-restricted value`-классов.

Интересно, что новая возможность затрагивает исключительно компилятор Java – JVM уже и так давно поддерживает байткод, в котором присутствуют инструкции перед вызовом `super()` или `this()`, если эти инструкции не трогают конструируемый объект (JVM даже ещё более либеральна, например, она разрешает несколько вызовов конструкторов, если любой путь обязательно завершается одним вызовом конструктора).

Primitive Types in Patterns, instanceof, and switch (Third Preview) (JEP 507)

Примитивные типы в паттернах, `instanceof` и `switch`, которые были в режиме preview в Java 23 и Java 24, остаются на третье preview без изменений:

```
// --enable-preview --source 25

Object obj = 42;
if (obj instanceof int i) { // matches
    System.out.println("int: " + i);
}

switch (obj) {
    case int i -> System.out.println("int: " + i); // matches
    case double d -> System.out.println("double: " + d);
    default -> System.out.println("other");
}
```

[Объяснить с](#) 

Проверять можно также и то, попадают ли значения в диапазон типа:

```
int i = 42;
if (i instanceof byte b) { // matches
    System.out.println("byte: " + b);
}

double d = 3.0;
switch (d) {
    case int i -> System.out.println("int: " + i); // matches
    case float f -> System.out.println("float: " + f);
    default -> System.out.println("other");
}
```

[Объяснить с](#) 

В примерах выше `42` попадает в диапазон `byte` (`[-128; 127]`), а `3.0` без потери точности приводится к `int`. Таким образом, это позволит более безопасно приводить одни числовые типы к другим, не прибегая к ручным проверкам диапазонов.

Подобные проверки могут быть полезны и в паттернах записей:

```
record JsonNumber(double d) {}

var json = new JsonNumber(3.0);
if (json instanceof JsonNumber(int i)) { // matches
    // ...
}
```

[Объяснить с](#) 

Если до Java 23-25 типы выражений-селекторов в `switch` могли быть только `int`, `short`, `byte` и `char` и для них поддерживались только константные ветки (`case 3` и т.п.), то сейчас

поддерживаются все примитивные типы и ветки могут быть паттернами:

```
float f = 1.0f;
switch (f) {
    case 0f -> System.out.println("0");
    case float x when x == 1f -> System.out.println("1"); // matches
    case float x -> System.out.println("other");
}

boolean b = "hello".isEmpty();
switch (b) {
    case true -> System.out.println("empty");
    case false -> System.out.println("non-empty"); // matches
}
```

[Объяснить с](#) 

API

Stable Values (Preview) (JEP 502)

В Java в режиме preview появилось новое API для *стабильных значений*. С помощью них можно создавать иммутабельные данные с отложенной инициализацией, то есть в момент первого обращения. При этом они имеют такую же производительность, как и `final` поля.

Вспомним, как в Java можно реализовать отложенную инициализацию классическими средствами:

```
class OrderController {
    private Logger logger = null;

    Logger getLogger() {
        if (logger == null) {
            logger = Logger.create(OrderController.class);
        }
        return logger;
    }

    void submitOrder(User user, List<Product> products) {
        getLogger().info("order started");
        ...
        getLogger().info("order submitted");
    }
}
```

[Объяснить с](#) 

В примере выше объект `logger` инициализируется в момент первого обращения. У такого подхода есть несколько проблем:

- Любой доступ к полю `logger` должен происходить через метод `getLogger()`. Это можно забыть сделать.

- Код не является потокобезопасным: объект `logger` может инициализироваться несколько раз.
- Компилятор не может применить оптимизацию `constant folding`, так как поле `logger` не является `final`.

Частично проблем выше можно избежать, прибегнув к другим более сложным идиомам, например, `double-checked locking` или `class holder`. Однако с `double-checked locking` код становится невероятно громоздким и хрупким (например, можно забыть вставить ключевое слово `volatile`), а так же отсутствует `constant folding`. С `class holder` код становится более-менее простым и надёжным (и есть `constant folding`), но у этой идиомы есть серьёзные ограничения: она применима только к статическим полям и для каждого поля приходится объявлять свой собственный класс. Также можно использовать `ConcurrentHashMap`, однако и у неё есть недостатки: отсутствует `constant folding` и есть проблемы, если функция возвращает `null`.

Теперь посмотрим, как код будет выглядеть с новым интерфейсом `StableValue`:

```
// --enable-preview --source 25
class OrderController {
    private final StableValue<Logger> logger = StableValue.of();

    Logger getLogger() {
        return logger.orElseSet(() -> Logger.create(OrderController.class));
    }

    void submitOrder(User user, List<Product> products) {
        getLogger().info("order started");
        ...
        getLogger().info("order submitted");
    }
}
```

[Объяснить с](#) 

Метод `orElseSet()` вызывается, чтобы получить содержимое объекта `logger`. Если содержимое уже получено, то оно просто возвращается. Если нет, то содержимое вычисляется путём вызова переданного `Supplier` 'a'. `StableValue` гарантирует, что `Supplier` вызовется не более одного раза, тем самым обеспечивая потокобезопасность.

Под капотом `StableValue` реализован таким образом, что использует внутреннюю для JDK аннотацию `@Stable` для хранения содержимого в поле, не являющегося `final`. Эта аннотация даёт сигнал виртуальной машине, что поле не будет меняться более одного раза, а значит виртуальная машина после установки может считать его константным значением, что открывает возможность для `constant folding`. Таким образом, `StableValue` позволяет добиваться одновременно гибкости инициализации и хорошей производительности.

Часто удобно создать стабильное значение и сразу же в месте объявления передать `Supplier` для инициализации. Для этого есть метод `StableValue.supplier()`:

```
// --enable-preview --source 25
class OrderController {
    private final Supplier<Logger> logger
```

```

        = StableValue.supplier(() -> Logger.create(OrderController.class));

    void submitOrder(User user, List<Product> products) {
        logger.get().info("order started");
        ...
        logger.get().info("order submitted");
    }
}

```

Объяснить с 

В примере выше объект `StableValue<Logger>` заменился на `Supplier<Logger>`, а код стал короче, потому что исчез метод `getLogger()`.

API также позволяет создавать не только значения с единичным содержимым, но и различные стабильные коллекции и стабильные функции. Приведём пример стабильного списка:

```

// --enable-preview --source 25
class Application {
    private static final List<OrderController> ORDERS
        = StableValue.list(POOL_SIZE, _ -> new OrderController());

    public static OrderController orders() {
        long index = Thread.currentThread().threadId() % POOL_SIZE;
        return ORDERS.get((int) index);
    }
}

```

Объяснить с 

В примере выше список `ORDERS` — это список, который для каждого индекса вычисляет значение в момент обращения и не более одного раза. Таким образом, `StableValue` — это ещё и хороший вариант для написания кэшей.

Remove the 32-bit x86 Port (JEP 503)

32-битный x86 порт OpenJDK был окончательно удалён. Это означает, что из кодовой базы удалены части кода, отвечающие за 32 бит x86. Собрать JDK под эту платформу больше нельзя.

Ранее 32-битный x86 порт был помечен как [deprecated for removal](#) в Java 24. Причинами удаления порта являются сложность поддержки, отсутствие на ней эффективной реализации виртуальных потоков, окончание поддержки [Windows 10 в октябре 2025](#) (последняя версия Windows, поддерживающая 32 бит), скорое [прекращение поддержки 32 бит x86 в Debian](#) и др. Удаление 32 бит x86 поможет упростить и ускорить разработку OpenJDK, в частности таких API как Loom, Foreign Function & Memory API, Vector API и т.д.

Structured Concurrency (Fifth Preview) (JEP 505)

Structured Concurrency, которое было в режиме preview в [Java 21](#), [Java 22](#), [Java 23](#) и [Java 24](#), остаётся в режиме preview в пятый раз.

По сравнению с Java 24 есть важное изменение в API: `StructuredTaskScope` теперь создаётся не через публичные конструкторы, а через различные перегрузки статического метода `open()`.

Structured Concurrency – это подход многопоточного программирования, который заимствует принципы из однопоточного структурного программирования. Главная идея такого подхода заключается в следующем: если задача расщепляется на несколько конкурентных подзадач, то эти подзадачи воссоединяются в блоке кода главной задачи. Все подзадачи логически сгруппированы и организованы в иерархию. Каждая подзадача ограничена по времени жизни областью видимости блока кода главной задачи.

В центре нового API класс `StructuredTaskScope`, у которого есть два главных метода:

- `fork()` – создаёт подзадачу и запускает её в новом виртуальном потоке,
- `join()` – ждёт, пока не завершатся все подзадачи или пока `scope` не будет [закрыт](#).

Пример использования `StructuredTaskScope`, где показана задача, которая параллельно запускает две подзадачи и дожидается результата их выполнения:

```
// --enable-preview --source 25
try (var scope = StructuredTaskScope.open()) {
    Subtask<String> user = scope.fork(() -> findUser());
    Subtask<Integer> order = scope.fork(() -> fetchOrder());

    scope.join(); // Join subtasks, propagating exceptions

    // Both subtasks have succeeded, so compose their results
    return new Response(user.get(), order.get());
}
```

[Объяснить](#) с 

Может показаться, что в точности аналогичный код можно было бы написать с использованием классического `ExecutorService` и `submit()`, но у `StructuredTaskScope` есть несколько принципиальных отличий, которые делают код безопаснее:

- Время жизни всех потоков подзадач ограничено областью видимости блока `try-with-resources`. Метод `close()` гарантированно не завершится, пока не завершатся все подзадачи.
- Если одна из операций `findUser()` и `fetchOrder()` завершается ошибкой, то другая операция отменяется автоматически, если ещё не завершена (в случае использования дефолтного `Joiner 'a` `awaitAllSuccessfulOrThrow()`, но возможны другие с другим поведением).
- Если главный поток прерывается в процессе ожидания `join()`, то обе операции `findUser()` и `fetchOrder()` отменяются при выходе из блока.
- В дампе потоков будет видна иерархия: потоки, выполняющие `findUser()` и `fetchOrder()`, будут отображаться как дочерние для главного потока.

Structured Concurrency должно облегчить написание безопасных многопоточных программ благодаря знакомому структурному подходу.

В Java 26 возможно будет [шестое preview](#) Structured Concurrency с мелкими изменениями в API.

Scoped Values (JEP 506)

Scoped Values, которые были в preview в Java 21, Java 22, Java 23 и Java 24, стали постоянным API.

По сравнению с Java 24 есть мелкое изменение: метод `ScopedValue.OrElse()` больше не принимает `null` в качестве аргумента.

Класс `ScopedValue` позволяет обмениваться иммутабельными данными без их передачи через аргументы методов. Он является альтернативой существующему классу `ThreadLocal`.

Классы `ThreadLocal` и `ScopedValue` похожи тем, что решают одну и ту же задачу: передать значение переменной в рамках одного потока (или дерева потоков) из одного места в другое без использования явного параметра. В случае `ThreadLocal` для этого вызывается метод `set()`, который кладёт значение переменной для данного потока, а потом метод `get()` вызывается из другого места для получения значения переменной. У данного подхода есть ряд недостатков:

- Неконтролируемая мутабельность (`set()` можно вызвать когда угодно и откуда угодно).
- Неограниченное время жизни (переменная очистится, только когда завершится исполнение потока или когда будет вызван `ThreadLocal.remove()`, но про него часто забывают).
- Высокая цена наследования (дочерние потоки всегда вынуждены делать полную копию переменной, даже если родительский поток никогда не будет её изменять).

Эти проблемы усугубляются с появлением виртуальных потоков, которые могут создаваться в гораздо большем количестве, чем обычные.

`ScopedValue` лишён вышеперечисленных недостатков. В отличие от `ThreadLocal`, `ScopedValue` не имеет метода `set()`. Значение ассоциируется с объектом `ScopedValue` путём вызова другого метода `where()`. Далее вызывается метод `run()`, на протяжении которого это значение можно получить (через метод `get()`), но нельзя изменить. Как только исполнение метода `run()` заканчивается, значение отвязывается от объекта `ScopedValue`. Поскольку значение не меняется, решается и проблема дорогого наследования: дочерним потокам не надо копировать значение, которое остаётся постоянным в течение периода жизни.

Пример использования `ScopedValue`:

```
private static final ScopedValue<FrameworkContext> CONTEXT = ScopedValue.newInstance();

void serve(Request request, Response response) {
    var context = createContext(request);
    ScopedValue.where(CONTEXT, context)
        .run(() -> Application.handle(request, response));
}

public PersistedObject readKey(String key) {
    var context = CONTEXT.get();
    var db = getDBConnection(context);
    db.readKey(key);
}
```

Объяснить с 

В целом `ScopedValue` является предпочтительной заменой `ThreadLocal`, т.к. навязывает

разработчику безопасную однонаправленную модель работы с неизменяемыми данными. Однако такой подход не всегда применим для некоторых задач, и для них `ThreadLocal` может быть единственно возможным решением.

Key Derivation Function API (JEP 510)

Key Derivation Function API, которое появилось в [Java 24 в режиме preview](#), стало постоянным API. Оно находится в пакете `javax.crypto`.

Функции выведения ключа (KDF - Key Derivation Functions) могут использоваться для вывода криптографически сильных секретных ключей (например, AES) на основе материала ключа (например, пароля) и других данных (например, соли).

Новое KDF API является гораздо более подходящим для задач выведения ключей, чем старое API на основе классов `KeyGenerator` и `SecretKeyFactory`.

Пока что единственной реализацией KDF API в JDK является HKDF (HMAC-based Extract-and-Expand Key Derivation Function), но в будущем планируется реализовать и другие KDF, например, [Argon2](#).

Пример выведения секретного AES-ключа с использованием HKDF:

```
// Create a KDF object for the specified algorithm
KDF hkdf = KDF.getInstance("HKDF-SHA256");

// Create an ExtractExpand parameter specification
AlgorithmParameterSpec params =
    HKDFParameterSpec.ofExtract()
        .addIKM(initialKeyMaterial)
        .addSalt(salt).thenExpand(info, 32);

// Derive a 32-byte AES key
SecretKey key = hkdf.deriveKey("AES", params);

// Additional deriveKey calls can be made with the same KDF object
```

[Объяснить с](#) 

Ранее в Java 21 появилось API для [механизма инкапсуляции ключей](#) (KEM). Вместе с KDF эти два API являются важными шагами для поддержки в Java [Hybrid Public Key Encryption](#) (HPKE), криптографической схемы, устойчивой к [квантовым атакам](#).

PEM Encodings of Cryptographic Objects (Preview) (JEP 470)

В режиме preview появилось новое API для кодирования криптографических объектов в [формат PEM](#) и декодирования обратно. Криптографическими объектами могут быть самые разные сущности: открытые ключи, закрытые ключи, сертификаты и т.д.

Приведём пример открытого ключа, закодированного в формате PEM:

```
-----BEGIN PUBLIC KEY-----
MFkwEwYHKoZIzj0CAQYIKoZIzj0DAQcDQgAEi/kRGOL7wCPTN4KJ2ppeSt5UYB6u
cPjjukDtFTXbguOIFDdZ65O/8HTUqS/sVzRF+dg7H3/tkQ/36KdtuADbwQ==
```

```
-----END PUBLIC KEY-----
```

[Объяснить](#) с 

Такой ключ можно декодировать с помощью нового класса `java.security.PEMDecoder` :

```
// --enable-preview --source 25
PEMDecoder decoder = PEMDecoder.of();
PublicKey key = (PublicKey) decoder.decode(data);
System.out.println(key);
```

[Объяснить](#) с 

Кодирование происходит с помощью класса `java.security.PEMEncoder` :

```
// --enable-preview --source 25
PEMEncoder encoder = PEMEncoder.of();
String data = encoder.encodeToString(key);
System.out.println(data);
```

[Объяснить](#) с 

Список всех криптографических объектов, которые можно кодировать/декодировать, лимитирован наследниками нового `sealed` интерфейса `java.security.DEREncodable` :

```
public sealed interface DEREncodable permits AsymmetricKey, KeyPair,
    PKCS8EncodedKeySpec, X509EncodedKeySpec, EncryptedPrivateKeyInfo,
    X509Certificate, X509CRL, PEMRecord {
}
```

[Объяснить](#) с 

Среди наследников выделяется особенный класс `java.security.PEMRecord` . Этот класс может содержать в себе любые PEM-данные. Он может пригодиться, когда для криптографического объекта в Java нет соответствующего API (например, [запрос сертификата PKCS #10](#)):

```
public record PEMRecord(String type, String content, byte[] leadingData)
    implements DEREncodable {
    ...
}
```

[Объяснить](#) с 

```
PEMRecord pr = PEMDecoder.of().decode(pem, PEMRecord.class);
```

[Объяснить](#) с 

В Java 26 будет [второе preview](#) этого API с некоторыми изменениями (например, `PEMRecord` будет переименован в `PEM`).

Vector API (Tenth Incubator) (JEP 508)

Векторное API в модуле `jdk.incubator.vector`, которое появилось ещё аж в [Java 16](#), остаётся в инкубационном статусе в десятый раз с некоторыми изменениями.

Векторное API остаётся так долго в инкубаторе, потому что зависит от некоторых фич проекта [Valhalla](#) (главным образом, от [value-классов](#)), который пока что находится в разработке. Как только эти фичи станут доступны в виде preview, векторное API сразу же перейдёт из инкубатора в статус preview.

JVM

Compact Object Headers (JEP 519)

Компактные заголовки объектов, которые появились в [Java 24](#) в качестве экспериментального режима, больше не являются экспериментальными. Они всё ещё не включены по умолчанию. Таким образом, чтобы их включить, теперь нужна только одна опция:

```
$ java -XX:+UseCompactObjectHeaders ...
```

[Объяснить с](#) 

При указании данной опции размер заголовков объектов в JVM уменьшается с 96/128 бит до 64 бит на 64-битных платформах. Компактные заголовки не только уменьшают размер кучи, но и могут улучшить производительность благодаря более высокой скорости выделения новых объектов, более низкой нагрузке на GC и лучшей локальности данных.

Сжатие заголовков достигается за счёт объединения mark-слова (64 бит) и class-слова (64 или 32 бит, если включены сжатые указатели на классы) в одно 64-битное слово. В новой схеме указатели на классы всегда являются сжатыми, и количество бит для них уменьшается с 32 до 22. Identity хеш-код остаётся неизменным: 31 бит. Количество тег-битов становится на один больше (для GC self forwarding). Битов для возраста GC остаётся 4, как и было. Также 4 бита резервируются на будущее для [Valhalla](#).

Работа по сжатию заголовков в OpenJDK ведётся в проекте [Lilliput](#), инициированным в [марте 2021 года](#). В дальнейшем возможно ещё большее сжатие заголовков до 32 бит, что уменьшит потребление памяти ещё сильнее.

JFR CPU-Time Profiling (Experimental) (JEP 509)

В JDK Flight Recorder появилось профилирование времени CPU в экспериментальном режиме (только на Linux). Для этого появилось новое событие `jdk.CPUTimeSample`:

```
$ java -XX:StartFlightRecording=jdk.CPUTimeSample#enabled=true,filename=profile.jfr ...
```

[Объяснить с](#) 

Профилирование времени CPU похоже на обычное профилирование времени выполнения, но с важной разницей: первое измеряет фактическое время, потраченное CPU, то есть исключая различные ожидания (например, чтения из сокета). Построив такой профиль, можно понять, какие методы больше всего потребляют CPU. Профилирование времени CPU в JFR использует специальный таймер в ядре Linux, который генерирует сигнал через каждый фиксированный промежуток потраченного времени CPU.

Несмотря на то, что фича является экспериментальной, для её включения не требуется флаг `-XX:+UnlockExperimentalVMOptions`.

JFR Cooperative Sampling (JEP 518)

JDK Flight Recorder теперь более стабильно выполняет снятие стек-трейсов потоков во время семплирования.

Чтобы точно снять стек-трейс, необходимо это делать во время сейфпоинта, иначе профилирование будет страдать от проблемы `safe point bias` и давать неточный результат. Поэтому раньше JFR снимал стек-трейсы в произвольных точках, а чтобы корректно построить стек-трейс, использовались эвристики. Однако эти эвристики могли быть неэффективными и даже приводить к крашам JVM.

Поэтому механизм семплирования был переработан и стал более стабилен. Теперь снятие стек-трейсов происходит только в сейфпоинтах, а проблема `safe point bias` решается кооперативно: вместо снятия стек-трейса в произвольном месте (не в сейфпоинте) семплирующий поток записывает запрос на семпл в локальную очередь целевого потока, а тот, доходя до сейфпоинта, достаёт запрос из очереди, реконструирует стек-трейс и выдаёт событие JFR.

Новый механизм всё ещё имеет некоторые недостатки. Например, когда целевой поток выполняет нативный код, новый подход не используется и применяется старый подход.

JFR Method Timing & Tracing (JEP 520)

Ещё одним новшеством в JDK Flight Recorder стали средства для тайминга и трассировки методов с помощью инструментации байткода.

В отличие от семплирующих профилировщиков (вроде `async-profiler`), которые периодически снимают стек-трейсы и записывают статистику для наиболее часто исполняемых методов, новый механизм в JFR позволяет сохранять полную и точную информацию для любых методов, даже если метод был вызван всего один раз и работал очень короткое время.

Тайминг и трассировка методов сохраняются с помощью двух новых событий `jdk.MethodTiming` и `jdk.MethodTrace`. Оба эти события принимают фильтр, в котором необходимо прописать метод (или методы).

В следующем примере происходит трассировка метода `HashMap.resize()` и дальнейшая распечатка через `jfr` всех его вызовов с подробной информацией (время начала вызова, длительность вызова, поток и стек-трейс):

```
$ java -XX:StartFlightRecording:method-trace=java.util.HashMap::resize,filename=recording.jfr
$ jfr print --events jdk.MethodTrace --stack-depth 20 recording.jfr
jdk.MethodTrace {
  startTime = 00:39:26.379 (2025-03-05)
  duration = 0.00113 ms
  method = java.util.HashMap.resize()
  eventThread = "main" (javaThreadId = 3)
  stackTrace = [
    java.util.HashMap.putVal(int, Object, Object, boolean, boolean) line: 636
    java.util.HashMap.put(Object, Object) line: 619
    sun.awt.AppContext.put(Object, Object) line: 598
    sun.awt.AppContext.<init>(ThreadGroup) line: 240
```

```
sun.awt.SunToolkit.createNewAppContext(ThreadGroup) line: 282
sun.awt.AppContext.initMainAppContext() line: 260
sun.awt.AppContext.getAppContext() line: 295
sun.awt.SunToolkit.getSystemEventQueueImplPP() line: 1024
sun.awt.SunToolkit.getSystemEventQueueImpl() line: 1019
java.awt.Toolkit.getEventQueue() line: 1375
java.awt.EventQueue.invokeLater(Runnable) line: 1257
javax.swing.SwingUtilities.invokeLater(Runnable) line: 1415
java2d.J2Ddemo.main(String[]) line: 674
]
}
...
```

[Объяснить с](#) 

Трассировка происходит путём инструментации метода `HashMap.resize()` и внедрения в него эмиссии события `jdk.MethodTrace`.

Рассмотрим другой пример, где замеряется тайминг всех статических инициализаторов:

```
$ java '-XX:StartFlightRecording:method-timing=::<clinit>,filename=clinit.jfr' ...
$ jfr view method-timing clinit.jfr
```

Method Timing

Timed Method	Invocations	Average Time
-----	-----	-----
sun.font.HBShaper.<clinit>()	1	32.500000 ms
java.awt.GraphicsEnvironment\$LocalGE.<clinit>()	1	32.400000 ms
java2d.DemoFonts.<clinit>()	1	21.200000 ms
java.nio.file.TempFileHelper.<clinit>()	1	17.100000 ms
sun.security.util.SecurityProviderConstants.<clinit>()	1	9.860000 ms
java.awt.Component.<clinit>()	1	9.120000 ms
sun.font.SunFontManager.<clinit>()	1	8.350000 ms
sun.java2d.SurfaceData.<clinit>()	1	8.300000 ms
java.security.Security.<clinit>()	1	8.020000 ms
sun.security.util.KnownOIDs.<clinit>()	1	7.550000 ms
...		

[Объяснить с](#) 

Здесь в качестве фильтра был указан `::<clinit>`, что означает все статические инициализаторы всех классов, а не один статический инициализатор конкретного класса.

В фильтрах могут быть указаны не только методы, но и классы. В таком случае замеряться тайминг или записываться трассировка будет для всех методов класса. Также можно указывать аннотации. В этом случае будут фильтроваться методы или классы, помеченные указанными аннотациями. Например, если указать фильтр `method-timing=@jakarta.ws.rs.GET`, то будет записываться статистика вызова эндпоинтов REST-сервиса. Или можно объявить свою аннотацию, например, `@StopWatch` и явно пометить ею методы, которые необходимо измерить:

```
$ java -XX:StartFlightRecording:method-timing=@com.example.StopWatch ...
```

[Объяснить с](#) 

Конфигурацию можно осуществлять не только через командную строку, но и через конфигурационные файлы:

```
<!-- timing.jfc -->
<?xml version="1.0" encoding="UTF-8"?>
  <configuration version="2.0">
    <event name="jdk.MethodTiming">
      <setting name="enabled">true</setting>
      <setting name="filter">
        com.example.Foo::method1;
        com.example.Bar::method2;
        ...
        com.example.Baz::method17
      </setting>
    </event>
  </configuration>
```

[Объяснить с](#) 

```
$ java -XX:StartFlightRecording:settings=timing.jfc,settings=default ...
```

[Объяснить с](#) 

Также инициировать процесс записи можно на уже запущенной JVM (через `jcmd`). Кроме того, через JMX трассировку и тайминг можно передавать по сети.

Ahead-of-Time Command-Line Ergonomics (JEP 514)

Теперь стало проще создавать AOT-кэши, поддержка которых появилась в [Java 24](#). Теперь это делается в один этап, а не в два.

В Java 24 AOT-кэш приходилось создавать двумя запусками, сначала записывая AOT-конфигурацию, а потом используя эту конфигурацию для создания кэша:

```
$ java -XX:AOTMode=record -XX:AOTConfiguration=app.aotconf \
  -cp app.jar com.example.App ...
$ java -XX:AOTMode=create -XX:AOTConfiguration=app.aotconf \
  -XX:AOTCache=app.aot -cp app.jar
```

[Объяснить с](#) 

Теперь же второй запуск не требуется, и создать кэш можно сразу же. Для этого появился новый ключ `AOTCacheOutput`:

```
$ java -XX:AOTCacheOutput=app.aot -cp app.jar com.example.App ...
```

[Объяснить с](#) 

Запуск приложения с кэшем проходит точно так же. Здесь ничего не изменилось:

```
$ java -XX:AOTCache=app.aot -cp app.jar com.example.App ...
```

[Объяснить с](#) 

Напомним, что Ahead-of-Time Class Loading & Linking – это технология, которая улучшает время старта Java-приложений благодаря использованию уже готовых загруженных и слинкованных классов. Создание таких классов происходит во время тренировочного запуска приложения, который создаёт ahead-of-time кэш. Далее все последующие ("боевые") запуски используют этот кэш для более быстрого старта.

Работа над ускорением запуска JVM ведётся в проекте [Leyden](#). Он был инициирован в [апреле 2020 года](#).

Ahead-of-Time Method Profiling (JEP 515)

Ещё одно улучшение AOT в Java – это появление профилей методов в AOT-кэшах. Профили методов собираются во время тренировочного запуска и в последующих запусках используются виртуальной машиной для более быстрого прогрева. Благодаря этому программа стартует быстрее и быстрее достигает пиковой производительности.

Например, небольшая программа, использующая Stream API и загружающая около 900 классов JDK, выполняется за 73 миллисекунды с использованием профиля, в то время как без профиля она выполняется за 90 миллисекунд. Таким образом, улучшение составляет 19%. При этом размер кэша увеличивается всего на 2.5%.

Профилирование методов необходимо для выявления горячих методов. Такие методы потребляют больше всего CPU, и чтобы программа работала быстро, JVM должна такие методы компилировать в нативный код в первую очередь. Однако профилирование – это довольно трудоёмкий процесс, особенно в фазе прогрева, когда профили ещё отсутствуют полностью. Если собирать профили заранее, то можно сократить некоторый объём работы на старте. Благодаря этому выигрышу фаза прогрева сокращается по времени.

Generational Shenandoah (JEP 521)

Режим работы с поколениями в сборщике мусора Shenandoah, который появился в [Java 24](#), больше не является экспериментальным. Теперь для его включения не нужен флаг `-XX:+UnlockExperimentalVMOptions`:

```
$ java -XX:+UseShenandoahGC -XX:ShenandoahGCMode=generational ...
```

[Объяснить с](#) 

Режим работы без поколений пока что остаётся режимом по умолчанию в Shenandoah.

Теги: [java](#), [java25](#), [jdk25](#), [jdk](#), [openjdk](#), [shenandoah](#), [leyden](#), [jfr](#), [aot](#)

Хабы: [Java](#)



Редакторский дайджест

Присылаем лучшие статьи раз в месяц

Подписаться

Оставляя почту, я принимаю [Политику конфиденциальности](#) и даю согласие на получение рассылок



2K+

111

Охват за 30 дней

Карма

Zheka Kozlov @orionll

Пользователь

Подписаться



X Telegram

Поток Бэкенд доступен 24/7 благодаря поддержке друзей Хабра

Хабр Карьера

КУРСЫ

для профессионального
роста и повышения
зарплаты

Хабр Курсы для бэкендеров

РЕКЛАМА

Практикум, Хекслет, SkyPro, авторские курсы — собрали всех и попросили скидки. Осталось выбрать!

Перейти

[Перейти в поток Бэкенд](#)

Комментарии 4

Другие новости



denis-19

16 минут назад

В ЦЕРН открыли исходный код своих библиотек компонентов KiCad

 2 мин  730 0 1 0 **anador**

1 час назад

401 meetup – регистрация участников

 2 мин  2.7K 0 0 0 **runaway_llm**

1 час назад

«Фабрика извлечения данных»: сотрудники Meta* восстали против слежки за мышью

 2 мин  3K +3 1 3 **honest_niceman**

1 час назад

В Kodacode добавили Claude Opus 4.7

 1 мин  2.2K +1 1 0 **maybe_elf**

1 час назад

SpaceX представила Starship V3

 4 мин  2.2K +12 1 2 **maybe_elf**

2 часа назад

Google представил ноутбуки Googlebook с Gemini и новой ОС

 2 мин  2.8K +7 1 0 **Clickru**
2 часа назад

Майские вебинары для маркетологов и бизнеса

 2 мин  1.9K 0 0 2 **LizzieSimpson**

2 часа назад

Авторы «ВКонтакте» с помощью «VK Доната» заработали 718 млн рублей за первый квартал года

🕒 1 мин 👁 2.3K

📌 +1

🔖 2

💬 2

 **Travis_Macrif**
2 часа назад

Sony представила линейку контроллеров Scuf Omega для PlayStation 5

🕒 2 мин 👁 3.1K

📌 +3

🔖 1

💬 5

 **maybe_elf**
2 часа назад

Tesla перенесла некоторые функции базового автопилота в платную версию FSD

🕒 2 мин 👁 2.5K

📌 +3

🔖 0

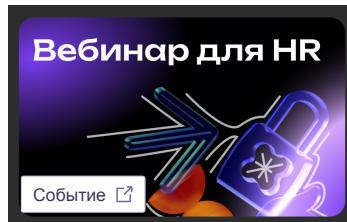
💬 2

Показать еще

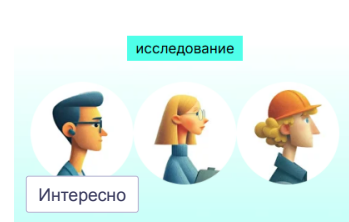
МИНУТОЧКУ ВНИМАНИЯ



Low-code и Big Data в фарме:
история про удалёнку без хаоса



Какие бенефиты нельзя резать,
даже при оптимизации бюджета



Лучшие практики по работе с
молодыми специалистами

ВАКАНСИИ

Java Developer

от 75 000 ₽ · ИТК academy · Казань · Можно удаленно

Java Developer

от 75 000 ₽ · ИТРУМ · Ростов-на-Дону · Можно удаленно

Java-разработчик в Маркет

от 180 000 до 350 000 ₽ · Яндекс · Москва

Java разработчик с нуля (стажер)

от 52 000 ₽ · Aston · Москва · Можно удаленно

Middle Java Developer

от 150 000 ₽ · ИТРУМ · Ростов-на-Дону

Больше вакансий на Хабр Карьере

ЧИТАЮТ СЕЙЧАС

Как Яндекс победил Google? Или почему блокировки — худший способ помочь отечественным технологиям

20K 133

Сколько я заработал в RuStore за 6 месяцев

7.5K 15

Я искал пассивный доход, открыл пункт выдачи Озон, ВБ и Яндекс, но оказался на работе, где часто не платят

52K 122

Мессенджер Max подписал стратегическое соглашение с операторами «большой четвёрки» («Билайн», МТС, «Мегафон» и Т2)

94K 176

Что на самом деле толкает заряд по проводу и создаёт ток?

8.3K 28

Как вернуть производительность цефа: перестаём гадать и разбираем узкие места

Турбо

ИСТОРИИ



Годнота из блогов компаний



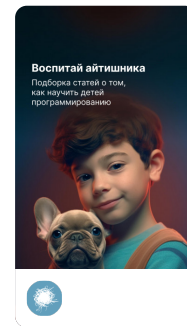
Большие данные, алгоритмы и другие секреты ретейла



Захлестнуло радиоволной



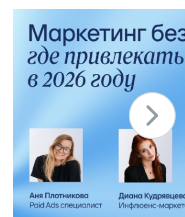
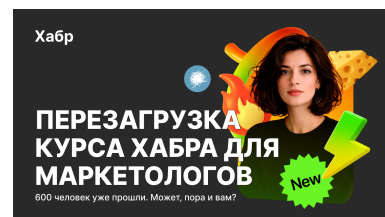
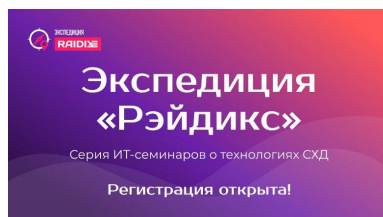
Схватил за мозг



Как учить детей программированию



БЛИЖАЙШИЕ СОБЫТИЯ



2 марта – 10 августа

PROBA Awards —
международная
коммуникационная
премия, учрежденная в
2000 году

Санкт-Петербург
Маркетинг

Больше событий в календаре

3 марта – 9 июня

Экспедиция «Рэйдикс»

Екатеринбург • Новосибирск •
Краснодар • Минск • Калининград •
Ростов-на-Дону
Разработка Другое

Больше событий в календаре

9 марта – 1 июня

Как продвигаться на
Хабре, если вы — не ИТ-
бренд?

Онлайн
Маркетинг

Больше событий в календаре

13 мая

Маркетинг б
где привлек
в 2026 году

Онлайн
Маркетинг

Больше событий в календаре

Ваш аккаунт

Войти
Регистрация

Разделы

Статьи
Новости
Хабы
Компании
Авторы
Песочница

Информация

Устройство сайта
Для авторов
Для компаний
Документы
Соглашение
Конфиденциальность

Услуги

Корпоративный блог
Медийная реклама
Нативные проекты
Образовательные программы
Стартапам



Настройка языка

Техническая поддержка

© 2006–2026, Habr